PROJECT REPORT

Automated Regression Testing of the SauceDemo Web Application

using Playwright and JavaScript

Submitted by: Manreet Gill

Roll No.: 24BCS10479

Course: BE-CSE

Date: 26 October 2026

Source code: https://github.com/gillmreet01/saucedemo-regression-tests

1. Introduction

Regression testing checks that features that worked before still work after the application changes. Doing this manually every time is slow and error-prone, so it is a good candidate for automation. This project builds an automated regression suite for SauceDemo (https://www.saucedemo.com), a public demo e-commerce website created for practising test automation. The suite is written in JavaScript using the Playwright Test framework.

2. Objectives

- Set up a clean, beginner-friendly Playwright + JavaScript project.
- Automate regression test cases for login, product listing and sorting, cart, checkout and logout.
- Use proper assertions to verify expected behaviour.
- Generate readable test reports, with screenshots, video and traces for failures.
- Keep the code simple so that it can be understood and explained easily.

3. Scope

3.1 In scope

- Login with valid, invalid, locked-out and empty credentials, and access protection of the inventory page.
- Product list display and two sort options (name A-Z, price low-high).
- Adding and removing items, cart badge, and cart page contents.
- Checkout form validation (first name, postal code), order overview and order completion.
- Logout.

3.2 Out of scope

- Performance, security and accessibility testing.
- Other demo users (problem_user, performance_glitch_user, error_user, visual_user).
- Cross-browser and mobile testing (only Chromium is configured).
- Product detail page, Continue Shopping and Cancel buttons, tax and total amounts.

4. Tools and Technologies

Tool	Purpose
JavaScript (Node.js)	Programming language and runtime
Playwright Test (@playwright/test)	Browser automation, test runner and assertions
Chromium	Browser used for execution
HTML reporter	Test report generation
VS Code, Git, GitHub	Development, version control and hosting

5. Project Structure

```
saucedemo-regression/
|-- playwright.config.js   settings: base URL, reporters, screenshots
|-- package.json           dependencies and npm scripts
|-- pages/                 Page Object classes
|   |-- LoginPage.js
|   |-- InventoryPage.js
|   |-- CartPage.js
|   `-- CheckoutPage.js
|-- tests/                 one test file per feature
|   |-- login.spec.js
|   |-- products.spec.js
|   |-- cart.spec.js
|   |-- checkout.spec.js
|   `-- logout.spec.js
|-- utils/
|   `-- testData.js        users, products, customer details
`-- docs/                  test plan and test cases
```

6. Design and Approach

6.1 Page Object Model

Each page of the application has a class in the pages folder. The class stores the locators of the elements on that page and the actions a user can perform (for example login(), addToCart(), sortBy()). The tests only call these methods. If the page layout changes, only the page class needs updating, which makes the suite easier to maintain.

6.2 Setup with beforeEach

Most tests need a logged-in user. Each spec file uses a beforeEach hook that opens the site and logs in with the standard user before every test. The checkout spec also adds the backpack to the cart and opens the checkout form. This keeps every test short and gives each one the same starting point.

6.3 Test data

Usernames, the password, product names and customer details are kept in utils/testData.js, so the tests contain no hard-coded values and data can be changed in one place.

6.4 Locators and assertions

Elements are located using stable data-test attributes. Assertions use Playwright web-first assertions such as toHaveURL, toHaveText, toContainText, toHaveCount and toBeVisible. These wait automatically (up to 5 seconds) until the condition is met, which avoids fixed sleeps and flaky tests.

6.5 Test independence

Playwright creates a fresh browser context for every test, so cookies, cart contents and sessions never leak between tests. This allows the suite to run in parallel.

7. Test Cases

A total of 17 test cases were automated. Each test case is identified by an ID in the test title.

ID	Feature	Scenario	Expected result	Status
TC01	Login	Valid user logs in	URL contains inventory.html; page title is "Products"	Pass
TC02	Login	Wrong password	Error says username and password do not match	Pass
TC03	Login	Locked-out user	Error says the user has been locked out	Pass
TC04	Login	Empty username	Error: "Username is required"	Pass
TC05	Login	Empty password	Error: "Password is required"	Pass
TC06	Login	Open inventory page without login	Error says you can only access it when logged in	Pass
TC07	Products	Product listing	Exactly 6 products are shown	Pass
TC08	Products	Sort by name A to Z	First product is "Sauce Labs Backpack"	Pass
TC09	Products	Sort by price low to high	First product is "Sauce Labs Onesie" (the cheapest, $7.99)	Pass
TC10	Cart	Add one item	Cart badge shows 1	Pass
TC11	Cart	Add two items	Cart badge shows 2	Pass
TC12	Cart	Remove item	Cart badge disappears	Pass
TC13	Cart	View cart	Cart page lists the added item	Pass
TC14	Checkout	Complete purchase end to end	Overview page opens; confirmation "Thank you for your order!"	Pass
TC15	Checkout	First name missing	Error: "First Name is required"	Pass
TC16	Checkout	Postal code missing	Error: "Postal Code is required"	Pass
TC17	Logout	User logs out	Returns to login page; Login button visible	Pass

8. Execution and Results

8.1 How to run

```
npm install
npx playwright install chromium
npm test
npm run report
```

8.2 Result summary

Item	Result
Browser	Chromium (headless)
Workers (parallel)	5
Total tests	17
Passed	17
Failed	0

Item	Result
Total execution time	18.6 seconds

All 17 tests passed against the live SauceDemo website. Screenshots of the run are shown in Section 9.

9. Screenshots

The screenshots below were taken while running the project on a Windows machine in VS Code.

9.1 Test execution in the terminal

npm test runs all 17 tests with 5 parallel workers. Every test shows a green tick and the summary reads "17 passed (18.6s)".

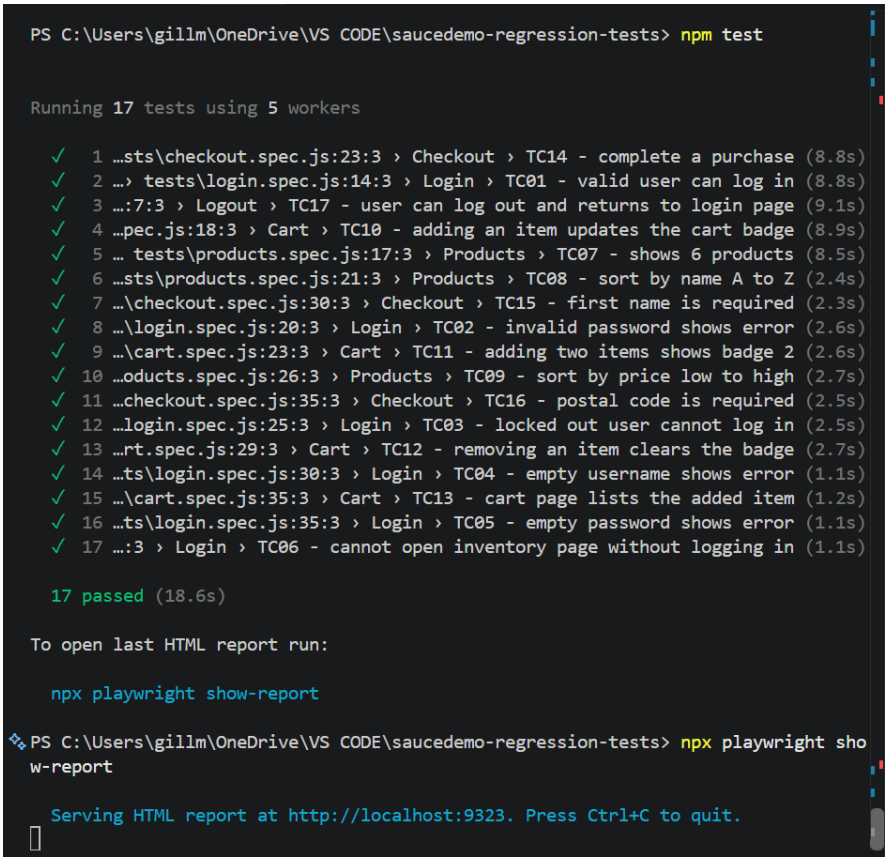


Figure 1: Terminal output of npm test, 17 passed

9.2 Generating the HTML report

After the run, npm run report starts a local server and serves the HTML report at http://localhost:9323. The report shows All 17, Passed 17, Failed 0, Flaky 0, Skipped 0, with the run time of 18.6 s and each test with its duration.

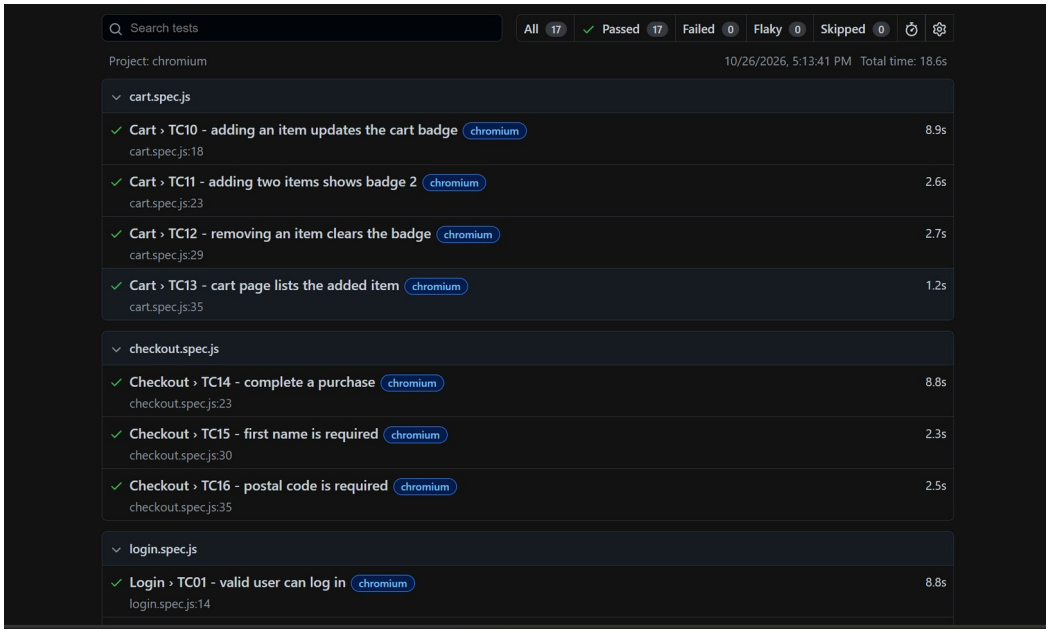


Figure 2: Playwright HTML report at localhost:9323 (17 passed, 0 failed, 18.6 s)

10. Reporting

The configuration uses two reporters. The list reporter prints each test result live in the terminal. The HTML reporter creates the playwright-report folder; it is opened with npm run report and shows every test with its status, duration and steps. For failed tests, a screenshot, a video and a trace are retained automatically in the test-results folder to help find the cause.

11. Challenges Faced

- Running npm commands from the wrong folder (the zip created a nested folder), which gave a package.json not found error. Solved by moving into the inner project folder.
- Choosing reliable locators. Solved by using data-test attributes.
- Avoiding repeated login code in every test. Solved with a beforeEach hook that logs in first.
- Publishing to GitHub: a placeholder repository URL and an existing remote commit caused push errors. Solved by correcting the remote URL and force-pushing the project.

12. Conclusion

The project delivers a working automated regression suite for the main user journeys of SauceDemo. It is built with a clear structure (Page Objects, beforeEach setup, separate test data), uses proper assertions, and produces reports. The suite can be re-run with a single command after any change, which demonstrates the value of test automation.

13. Future Scope

- Run on Firefox and WebKit by adding projects to the configuration.
- Add more tests: sort Z to A and high to low, last name validation, cart reset after an order, and the other demo users such as problem_user and performance_glitch_user.
- Run the suite automatically on every push using GitHub Actions.

- Add visual comparison and API tests.

14. References

- Playwright documentation: https://playwright.dev
- SauceDemo application: https://www.saucedemo.com
- Project repository: https://github.com/gillmreet01/saucedemo-regression-tests